

บทที่ 4

แนวคิดเชิงวัตถุ

ในยุคต้น ภาษาที่ใช้สั่งการเป็นภาษาที่สั่งให้คอมพิวเตอร์ทำงานเป็นลำดับของคำสั่ง งานประยุกต์จึงใช้ภาษาเชิงลำดับสั่งงาน ต่อมาเมื่อพบว่างานที่สั่งให้คอมพิวเตอร์ทำมีความซับซ้อนมากขึ้น การสั่งงานทำให้วิธีการเขียนโปรแกรมขึ้นอยู่กับตัวบุคคล เพราะลำดับความคิดที่ถ่ายทอดมาเป็นรูปโปรแกรมนั้นยากที่จะทำความเข้าใจได้ โปรแกรมที่พัฒนาจึงขึ้นกับตัวบุคคล ไม่สามารถให้อีกบุคคลหนึ่งดำเนินการตรวจสอบหรือทำความเข้าใจเกี่ยวกับโปรแกรมได้ง่าย

จึงทำให้เกิดแนวความคิดการเขียนโปรแกรมเชิงวัตถุ (Object – Oriented Programming) ขึ้นมา โดยเป็นวิธีใหม่ในการพัฒนาซอฟต์แวร์ (Software) เพื่อเพิ่มประสิทธิภาพในการผลิตซอฟต์แวร์ รูปแบบเชิงวัตถุเป็นการแสดงให้เห็นองค์ประกอบสิ่งต่างๆ (object) ที่เราสนใจ ซึ่งจะมีข้อกำหนดที่กำหนดให้กับองค์ประกอบเหล่านั้น และแสดงความสัมพันธ์ระหว่างองค์ประกอบเหล่านั้น

4.1 หลักการของแนวความคิดเชิงวัตถุ จะประกอบไปด้วยองค์ประกอบเหล่านี้คือ

1. **Object and Object Identifier** หมายถึง ออปเจกต์ใดๆ ที่อยู่ในรูปแบบที่กำหนด และมีการกำหนดค่า id ที่ไม่ซ้ำกัน เพื่อใช้ในการอ้างถึงออปเจกต์นั้น
2. **Attributes and Methods:** ออปเจกต์ทุกๆ ออปเจกต์จะมีคุณลักษณะ (state) คือเป็นเซตของค่าข้อมูลของแอตทริบิวต์ของออปเจกต์ และการกระทำ (behavior) เป็นเซตของเมธอด ส่วนของโปรแกรมซึ่งมีการดำเนินการบนลักษณะของออปเจกต์ ซึ่งคุณลักษณะและการกระทำจะถูกห่อหุ้มไว้ในออปเจกต์ และจะถูกอ้างหรือเรียกให้ทำงานจากภายนอก โดยผ่านทางข่าวสาร (message)
3. **Class** หมายถึง การรวมกลุ่มของออปเจกต์ทั้งหมดที่มีลักษณะ และเมธอดที่เหมือนกัน ออปเจกต์หนึ่งๆ จะถูกสร้างขึ้นจากคลาสเพียงหนึ่งคลาส ซึ่งคลาสมีลักษณะเหมือนกับ abstract data type คลาสอาจจะเป็นข้อมูลพื้นฐานก็ได้ เช่น integer, string, Boolean เป็นต้น
4. **Class Hierarchy and Inheritance** เป็นการสร้างคลาสใหม่ (subclass) จากคลาสเดิม (superclass) ที่มีอยู่ก่อน คลาสที่สร้างขึ้นใหม่จะสืบทอดคุณลักษณะ และเมธอดทั้งหมดจากคลาสเดิม โดยคลาสใหม่สามารถที่จะเพิ่มคุณลักษณะ และเมธอดใหม่ เพิ่มเข้าไปได้ จะแบ่งเป็นสองประเภทคือ Single Inheritance และ Multiple Inheritance

4.1.1. Object Structure

แนวทางเชิงวัตถุมีพื้นฐานมาจากการห่อหุ้มข้อมูล และโปรแกรม (Encapsulating) ดังนั้นการติดต่อกันระหว่างออปเจกต์จะผ่านทางข่าวสาร (message) เสมอ ซึ่งโดยทั่วไปแล้วออปเจกต์จะเกี่ยวข้องกับ

- เซตของตัวแปรที่บรรจุข้อมูลของออปเจกต์
- เซตของข่าวสาร ซึ่งออปเจกต์จะตอบสนองต่อข่าวสารที่ได้รับจากภายนอก

- เซตของเมธอด ซึ่งแต่ละเมธอดจะเป็นโปรแกรมที่ดำเนินการกับข่าวสาร และส่งข้อมูลกลับ
 สิ่งที่ทำให้ต้องมีการใช้ข่าวสาร กับเมธอดระหว่างแอปเจคนั้น เนื่องจากในระบบเชิงวัตถุ เราสามารถ
 ที่จะแก้ไขนิยามของตัวแปร และเมธอด โดยไม่มีผลกระทบต่อระบบ ซึ่งคุณสมบัตินี้เป็นข้อดีของการเขียน
 โปรแกรมเชิงวัตถุ

4.1.2.Object Classes

โดยปกติ ในฐานข้อมูลจะมีแอปเจตที่คล้ายๆ กัน หมายความว่า แอปเจตมีการตอบสนองต่อข่าวสาร
 เดียวกัน โดยใช้เมธอดเดียวกัน และตัวแปรชื่อเดียวกัน ชนิดข้อมูลเดียวกัน ดังนั้นเราจึงจัดกลุ่มของแอปเจตที่มี
 ลักษณะคล้ายๆ กัน แล้วกำหนดเป็นคลาสๆ หนึ่ง และแต่ละแอปเจตที่ถกสร้างมาจากคลาสก็เรียกว่า เป็น
 Instance ของคลาส แอปเจตทุกตัวจะมีลักษณะเหมือนกัน แต่จะแตกต่างกันตรงข้อมูลที่ถูกระบุให้กับตัว
 แปรในแอปเจต

ตัวอย่าง เรานิยามคลาส Employee ในรูปแบบของคำสั่งจำลอง ซึ่งแสดงตัวแปรและเมธอด(ใน
 ตัวอย่างยังไม่มีการจัดการกับเมธอดของคลาส)

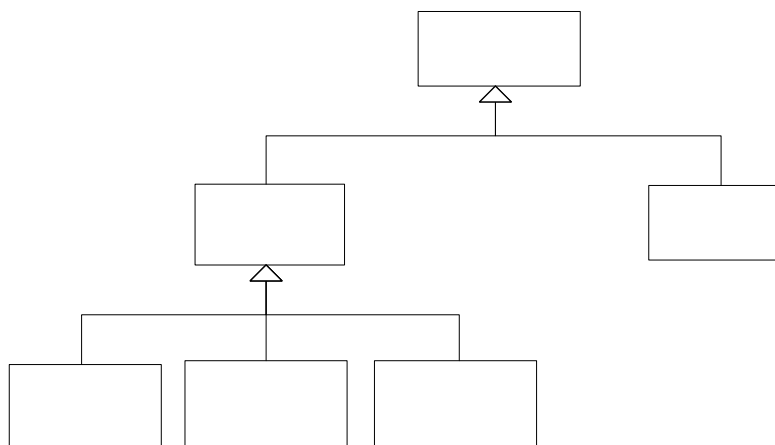
```
Class Employee {
    /* Variables */
    String name;
    String address;
    Date start_date;
    Integer salary;
    /* Methods */
    Integer annual_salary();
    String get_name();
    String get_address();
    Integer set_address(String new_address);
    Integer employment_length();
}
```

จากนิยามของคลาส Employee จะประกอบไปด้วยตัวแปร name และ address ซึ่งมีชนิดข้อมูลเป็น
 String start_date มีข้อมูลเป็น Date และ salary มีชนิดข้อมูลเป็น Integer แต่ละแอปเจตจะตอบสนองต่อ
 ข่าวสาร 5 ข่าวสาร คือ annual_salary, get_name, get_address, set_address และ employment_length โดยเมธ
 อดแต่ละอันมีการกำหนดชนิดข้อมูลไว้ด้านหน้า หมายถึงชนิดของการตอบสนองข้อมูลกลับไปยังเมธอดนั้น

4.1.3. Inheritance

ในระบบฐานข้อมูลเชิงวัตถุจะประกอบด้วยคลาสจำนวนมาก อย่างไรก็ตามคลาสหลายๆ คลาสจะมี
 ลักษณะคล้ายๆ กัน ตัวอย่างเช่น พนักงานของธนาคาร จะมีลักษณะที่คล้ายกับลูกค้า เช่น ชื่อ ที่อยู่ โทรศัพท์

เป็นต้น เพื่อแสดงให้เห็นถึงลักษณะที่คล้ายกันของคลาส เราจะแสดง E-R ไดอะแกรมในลักษณะของลำดับชั้นของ specialization (ความสัมพันธ์แบบ is a)



รูปที่ 4.1 แสดงลำดับชั้นของคลาสแบบ Specialization

ตัวอย่าง ลำดับชั้นของคลาสที่กำหนดเป็นคำสั่งจำลองได้ดังนี้

```

Class Person {
    String name;
    String address;
}

Class Customer isa Person {
    Integer credit_rating;
}

Class Employee isa Person {
    Date start_date;
    Integer salary;
}

Class Officer isa Employee {
    Integer officer_number;
    Integer expense_account_number;
}

Class Teller isa Employee {
    Integer hours_per_week;
    Integer station_number;
}

Class Secretary isa Employee {
    Integer hours_per_week;
  
```

```
String manager;
}
```

คำว่า “is a” ในคำสั่งด้านล่างใช้สำหรับบอกว่าคลาสนั้นมีลักษณะเช่นเดียวกันกับอีกคลาสหนึ่ง โดยเราเรียกคลาสที่ specialization ว่า Subclass ตัวอย่างเช่น Employee เป็น Subclass ของ Person และ Teller เป็น Subclass ของ Employee ในทางกลับกัน Employee ก็เป็น Superclass ของ Teller

ข้อดีของการสืบทอดคุณสมบัติในระบบเชิงวัตถุคือ มีคุณลักษณะของ code-reuse นั่นคือเมธอดใดๆ ของคลาส A สามารถที่จะถูกเรียกใช้โดยคลาส B ซึ่งเป็น Subclass A ได้ทันที ทำให้ไม่ต้องเขียนคำสั่งในคลาส B อีก

4.1.4. Multiple Inheritance

วิธีการ Multiple inheritance เป็นวิธีการที่ยอมให้คลาสหนึ่งสืบทอดคุณสมบัติ และเมธอดจากหลายๆ คลาสได้ แต่เมื่อเรามีการสืบทอดแบบหลายคลาส อาจ会有ความคลุมเครือเกิดขึ้นในกรณีที่ว่าตัวแปรหรือเมธอดที่สืบทอดมาจากหลายคลาสมีชื่อเดียวกัน ทำให้ Subclass เกิดความสับสนในการเรียกใช้งานตัวแปรหรือเมธอดดังกล่าว

4.1.5. Object Identity

Object identity: วัตถุจะยังคงรักษาความเป็น identity แม้ว่าข้อมูลของตัวแปรหรือเมธอดของวัตถุจะมีการเปลี่ยนแปลง แนวคิดของ identity นี้ไม่ได้มีอยู่ในฐานข้อมูลแบบเชิงสัมพันธ์ ซึ่งในระบบฐานข้อมูลเชิงสัมพันธ์นั้น เรคอร์ดของเทเบิลจะแตกต่างกันด้วยข้อมูลที่ถูเก็บอยู่

รูปแบบต่างๆ ของ identity

- **Value** หมายถึง ค่าของข้อมูลที่ใช้ในการ identity เช่น Primary key
- **Name** เป็นการกำหนดเพื่อใช้ในการ identity เช่น ชื่อแฟ้มในระบบแฟ้มข้อมูล
- **Built-in** เป็นการกำหนด identity โดยอัตโนมัติให้กับข้อมูล หรือภาษาโปรแกรม โดยผู้ที่ไม่จำเป็นต้องกำหนด identifier รูปแบบนี้ถูกใช้กับระบบเชิงวัตถุ โดยแต่ละวัตถุจะถูกกำหนด identifier โดยอัตโนมัติ เมื่อวัตถุถูกสร้างขึ้น

ในทางปฏิบัติ Object identity จะถูกกำหนดให้มีค่าไม่ซ้ำกัน เรียกว่า OID ซึ่งค่าของ OID นี้จะไม่สามารถมองเห็นได้โดยผู้ใช้ทั่วไป แต่จะถูกใช้โดยระบบเพื่อระบุวัตถุแต่ละวัตถุ

4.1.6. Encapsulation

วิธีการ Encapsulation เป็นวิธีการที่ช่วยปกป้องข้อมูลโดยไม่ให้อุปกรณ์อื่นเข้ามาทำการเปลี่ยนแปลงข้อมูลก่อนได้รับอนุญาต โดยการเข้าถึงตัวแปรของวัตถุนั้นจะต้องผ่านทางเมธอดเท่านั้น

4.1.7. Method Overriding and Polymorphism

วิธีการ Overriding เป็นวิธีการที่ Subclass สามารถสร้างเมธอดที่มีชื่อเดียวกันกับเมธอดที่สืบทอดมาจาก Superclass ได้ เพื่อให้ Subclass นั้นมีความเฉพาะเจาะจงมากขึ้น

การ Polymorphism เป็นการทำให้วัตถุที่ต่างกันสามารถตอบสนองต่อข่าวสารเดียวกันได้หลายๆ วิธีการ โดยในระบบเชิงวัตถุคำว่า “Polymorphism” จะหมายถึง

1. เมธอดสามารถใช้ชื่อเดียวกันได้ในหลายๆ คลาส
2. ผู้ใช้ส่งข่าวสาร(message) เดียวกัน ไปยังวัตถุจากคลาสต่างๆ กัน ก็ยังได้ผลลัพธ์ที่ถูกต้อง

4.1.8. Abstract Class

Abstract Class เป็นวิธีการที่ช่วยลดความซ้ำซ้อนของคุณสมบัติ โดยแยกเอาเฉพาะสิ่งที่ผู้ใช้งานสนใจออกมา โดยการแยกแยะเอกลักษณ์เป็นการแสดงคุณลักษณะที่สำคัญของวัตถุ ซึ่งเป็นคุณลักษณะที่ทำให้วัตถุนั้นๆ แตกต่างจากวัตถุอื่นๆ